Interview Preparation Guide

C++ · Low Latency · Python · OOP · Quant Fundamentals

Two-day revision plan

- How to use this guide
  - Two-day plan
  - The seven answers to rehearse out loud
  - What to skip
  - The thing that actually differentiates
- Part 1 — C++ Memory Model and Concurrency
  - 1.1 What a data race actually is
  - 1.2 `volatile` is not for threading
  - 1.3 Memory ordering
  - 1.4 Fences
  - 1.5 Mutexes and what they cost
  - 1.6 Atomics in practice
  - 1.7 Lock-free SPSC queue (sketch this on paper)
  - 1.8 Common interview questions
  - 2.1 The memory hierarchy
  - 2.2 Cache lines
  - 2.3 AoS vs SoA
  - 2.4 False sharing
  - 2.5 Container internals
  - 2.6 Move semantics
  - 2.7 RAII and smart pointers
  - 2.8 Undefined behaviour you should be able to list
  - 2.9 Common interview questions
  - 3.1 Templates: why HFT codebases use them
  - 3.2 Floating point
  - 3.3 Common interview questions
  - 4.1 The tick-to-trade path
  - 4.2 Kernel bypass
  - 4.3 The rules for a hot path
  - 4.4 System tuning
  - 4.5 Measurement
  - 4.6 Benchmarking traps
- Python
  - 4.7 The data model
  - 4.8 The GIL
  - 4.9 The `dict.get` vs `[]` trap
  - 4.10 NumPy
  - 4.11 pandas
  - 4.12 pybind11
  - 4.13 Common Python interview questions
  - 5.1 SOLID, briefly
  - 5.2 Composition over inheritance
  - 5.3 Virtual functions and destructors
  - 5.4 PIMPL
  - 5.5 Common design questions
- Quant Fundamentals

- 5.6 Options: the basics
- 5.7 Put-call parity
- 5.8 The Greeks
- 5.9 Implied volatility
- 5.10 P&L and exposure vocabulary
- 5.11 Execution quality
- 5.12 Indian market specifics
- 5.13 Common quant interview questions

How to use this guide

This is a **revision** document, not a textbook. It assumes you already write production C++ and is aimed at making what you know retrievable under pressure.

Topics are ordered by how likely they are to come up given the assessment you completed — threading, memory layout, floating point, and enough options fluency not to misuse a term.

Two-day plan

Day 1 — C++ (8 hours)

Block	Time	Content
1	2.0 h	Re-read your own submission, out loud. Rehearse the seven answers below.
2	2.5 h	Part 1 — memory model and concurrency
3	2.0 h	Part 2 — cache and memory layout
4	1.5 h	Part 2 — language mechanics; Part 3 — templates

Day 2 — Everything else (8 hours)

Block	Time	Content
5	2.0 h	Part 4 — low latency
6	1.5 h	Part 4 — Python
7	1.0 h	Part 5 — OOP
8	2.5 h	Part 5 — quant fundamentals
9	1.0 h	Two bug stories; “why leave your current role”

The night before: don’t study. Skim the seven answers and sleep. You will be reasoning live rather than reciting, and that degrades badly when tired.

The seven answers to rehearse out loud

These come directly from your submission. If any of them doesn’t come out fluently in your own words, that is where to spend time — ahead of anything else in this guide.

1. Why `p > 0.0` catches NaN, and why `p <= 0.0` would not.
2. Why NaN breaks `std::sort` — strict weak ordering violation is **UB**, not a bad order.
3. Why `-ffp-contract=off` matters, and why the plain build passes anyway.
4. Why the `+ size` in `(head + size - count + i) % size`.
5. Why a mutex gives exclusion but not **order** — 30 runs, 30 different answers.
6. Why parallelising that return loop is wrong — 1.7% of the work, Amdahl caps it at 1.017x.
7. Why `e^{+rT}` and not `e^{-rT}` in the synthetic forward.

What to skip

Template metaprogramming beyond `if constexpr`. Coroutines. Modules. Boost. Design patterns as a catalogue. Exception-safety guarantees in depth.

If asked about these, **"I haven't used that"** is a fine answer. Bluffing on C++ minutiae goes badly, because the interviewer usually knows.

The thing that actually differentiates

Everyone revises the same list. What almost nobody brings is a **measured** story: *"I found FMA contraction breaking Python parity on 1,781 of 3,000 fills, and here is the `vfmadd132sd` instruction that caused it."*

You have four or five of those. Depth on one real thing beats surface coverage of twenty.

Part 1 — C++ Memory Model and Concurrency

Highest-probability deep-dive topic. The assessment gave you threading problems in two out of five questions.

1.1 What a data race actually is

Basic. Two threads access the same memory location, at least one writes, and there is no synchronisation between them. That is a data race.

The part people get wrong: a data race is **undefined behaviour**, not "you might read a stale value." The standard gives your program no meaning at all. The compiler is allowed to assume races don't happen, so it may:

- hoist a load out of a loop (your `while (!stop)` never terminates)
- split a 64-bit write into two 32-bit writes
- reorder your writes so another thread sees them out of order
- delete code it proves unreachable under the no-race assumption

```
bool stop = false;          // plain bool, written by another thread

void worker() {
    while (!stop) {         // compiler may load `stop` ONCE, before the loop
        doWork();           // -> infinite loop even though stop becomes true
    }
}
```

Fix: `std::atomic<bool> stop`. This is a real bug I hit while writing the Problem 3 tests — ThreadSanitizer caught it.

Key sentence for interview: *"A data race is UB, so reasoning about 'what value will I see' is already the wrong question — the program has no defined behaviour at all."*

1.2 `volatile` is not for threading

Basic. `volatile` tells the compiler "don't optimise away accesses to this." It came from memory-mapped hardware registers.

What it does NOT give you:

	`volatile`	`std::atomic`
Prevents compiler removing the access	yes	yes
Atomicity (no torn reads/writes)	**no**	yes
Ordering with respect to other memory	**no**	yes
Prevents CPU reordering	**no**	yes

So `volatile int x` shared between threads is still a data race. In Java and C# `volatile` does imply ordering; in C and C++ it does not. That confusion is the source of most misuse.

Interview answer: *"`volatile` is for memory-mapped I/O. For threading it gives you neither atomicity nor ordering, so it's still a data race — you want `std::atomic`."*

1.3 Memory ordering

This is the topic most likely to be pushed on. Learn the five orderings by *what they guarantee*, not by name.

The problem

Both the compiler and the CPU reorder memory operations for speed. Single-threaded, you never notice — the result is "as if" sequential. Multi-threaded, another thread can observe the reordering.

```
// Thread A                    // Thread B
data = 42;                     if (ready) {
ready = true;                      use(data);   // might see data == 0!
                               }
```

Without ordering constraints, `ready = true` can become visible before `data = 42`.

The orderings

`memory_order_relaxed` Atomicity only. No ordering guarantees at all. Use when you only need the operation to be indivisible — a statistics counter is the classic case.

```
counter.fetch_add(1, std::memory_order_relaxed);   // just count, order irrelevant
```

`memory_order_release` (on a store) Everything written *before* this store, in program order, becomes visible to any thread that acquire-loads this same variable.

`memory_order_acquire` (on a load) Everything the releasing thread wrote before its release-store is visible to you after this load.

They work as a pair, on the same variable:

```
// Thread A (writer)
data = 42;                                      // ordinary write
ready.store(true, std::memory_order_release);   // publish

// Thread B (reader)
if (ready.load(std::memory_order_acquire)) {    // observe
    use(data);                                  // GUARANTEED to see 42
}
```

The mental model: **release = "everything I did is now visible"**, **acquire = "I can now see everything they did"**.

`memory_order_acq_rel` — for read-modify-write ops (`fetch_add`, `exchange`, `compare_exchange`). Acts as acquire for the read half and release for the write half.

`memory_order_seq_cst` — the default. Everything above, plus a single total order that all threads agree on. Strongest, and the only one where the intuitive "there's one global sequence of events" model holds.

What each costs

On x86-64:

Operation	Instruction
relaxed load/store	plain `mov`
acquire load	plain `mov`
release store	plain `mov`
seq_cst store	**`mfence` or `xchg` (~20–30 cycles)**
RMW (any ordering)	`lock`-prefixed instruction (~20–50 cycles)

x86 has a strong memory model — it won't reorder store-store or load-load. So acquire/release are *free* there, and only seq_cst costs anything.

On ARM (aarch64): weakly ordered, so acquire/release emit real instructions (`ldar`/`stlr`). Code that looks free on x86 costs on Graviton or Apple Silicon.

Interview answer: *"Default to seq_cst for correctness, drop to acquire/release when you've identified the pairing, and relaxed only when you genuinely need nothing but atomicity. On x86 acquire/release are free, so the seq_cst default is usually the only one you pay for."*

1.4 Fences

```
std::atomic_thread_fence(std::memory_order_release);
```

A standalone barrier, not attached to any variable. Used when you want to order several relaxed operations at once, as in a seqlock.

Two things to know:

1. On x86 a release or acquire fence emits **no instructions** — it's purely a compiler barrier.
2. **ThreadSanitizer cannot model standalone fences.** GCC and Clang emit `-Wtsan` and then reason unsoundly. So a fence-based algorithm cannot be validated with TSan; you need a model checker or a careful proof.

That second point is a strong thing to know, and it came directly out of the assessment.

1.5 Mutexes and what they cost

Basic. `std::mutex` gives mutual exclusion. `std::lock_guard` is RAII — locks in the constructor, unlocks in the destructor, so it can't be forgotten on an early return or an exception.

```
std::lock_guard<std::mutex> lock(mutex_);   // locked here
// ...                                       // unlocked automatically at scope exit
```

Type	Use
`lock_guard`	simplest, locks for the whole scope
`unique_lock`	can unlock early, defer, or transfer; needed for condition variables
`scoped_lock` (C++17)	locks multiple mutexes deadlock-free
`shared_lock`	reader lock for `shared_mutex`

Cost: uncontended lock/unlock is ~15–20 ns (it's just an atomic exchange plus a check). Contended is *unbounded* — you go to the kernel via futex and get descheduled.

Why `shared_mutex` is often a trap

It looks like the obvious choice for one-writer-many-readers. But acquiring a *shared* lock still **writes** to a shared counter. So N readers on N cores all ping-pong one cache line and serialise on coherence traffic — frequently slower than a plain mutex for short critical sections. It also has implementation-defined writer starvation behaviour.

Interview answer: *"`shared_mutex` only helps when the read critical section is long enough to amortise the atomic RMW*

on the shared counter. For a few-nanosecond read it's usually slower than a plain mutex, because readers now contend with each other."

The deeper point: a mutex gives exclusion, not order

This was the key finding in the assessment. If a computation is order-dependent, a mutex prevents corruption but does **not** make the result deterministic — whichever thread grabs the lock first wins, and that varies run to run.

Measured on the position tracker: the same fills across four threads under one mutex gave **30 distinct results from 30 identical runs**. Share counts agreed every time (addition commutes) while average prices did not.

The fix is usually structural, not more locking: shard by key so each key is owned by exactly one thread, which preserves order by construction and needs no lock at all.

1.6 Atomics in practice

Compare-and-swap

```
bool compare_exchange_weak(T& expected, T desired);
bool compare_exchange_strong(T& expected, T desired);
```

Both: "if the current value equals `expected`, set it to `desired` and return true; otherwise write the current value into `expected` and return false."

`weak` may fail **spuriously** (return false even when the values matched) because on some architectures it maps to LL/SC which can fail for unrelated reasons. It's cheaper, so use it inside a retry loop:

```
T old = value.load(std::memory_order_relaxed);
while (!value.compare_exchange_weak(old, old + 1)) {
    // old was updated with the current value; just retry
}
```

Use `strong` when you're **not** looping.

Lock-free vs wait-free

- **Lock-free**: at least one thread makes progress. Individual threads can starve.
- **Wait-free**: every thread completes in a bounded number of steps.

A single-producer ring buffer writer is wait-free. A CAS retry loop is lock-free but not wait-free.

The seqlock pattern

The design used in the assessment. Writer never blocks; readers retry if they collide:

```
// Writer
seq.store(WRITING, relaxed);                      // mark busy
std::atomic_thread_fence(memory_order_release);
// ... write payload ...
seq.store(version, memory_order_release);         // publish

// Reader
uint64_t s1 = seq.load(memory_order_acquire);
if (s1 == WRITING) retry;
// ... copy payload ...
std::atomic_thread_fence(memory_order_acquire);
if (seq.load(memory_order_relaxed) != s1) retry;  // torn -> discard
```

Why it scales: readers issue only loads. They never write shared state, so they don't contend with each other and can never stall the writer.

What it costs: readers can retry, and under an extreme write rate can give up. That's a regression in the *reader's* worst case, traded for the *writer* never blocking. For market data that's clearly right — but it's a decision to state, and the give-up count must be a monitored metric.

1.7 Lock-free SPSC queue (sketch this on paper)

The single most likely "write me a lock-free thing" question.

```
template <typename T, size_t N>   // N a power of two
class SpscQueue {
    static_assert((N & (N - 1)) == 0, "N must be a power of two");
public:
    bool push(const T& item) {
        size_t w = write_.load(std::memory_order_relaxed);   // only we write it
        size_t next = (w + 1) & (N - 1);
        if (next == read_.load(std::memory_order_acquire)) {
            return false;                                    // full
        }
        buffer_[w] = item;
        write_.store(next, std::memory_order_release);       // publish
        return true;
    }

    bool pop(T& out) {
        size_t r = read_.load(std::memory_order_relaxed);
        if (r == write_.load(std::memory_order_acquire)) {
            return false;                                    // empty
        }
        out = buffer_[r];
        read_.store((r + 1) & (N - 1), std::memory_order_release);
        return true;
    }
private:
    alignas(64) std::atomic<size_t> write_{0};   // separate cache lines,
    alignas(64) std::atomic<size_t> read_{0};    // or the two indices false-share
    T buffer_[N];
};
```

Points to make when you write it:

- Each index is written by **exactly one** thread, so its own load can be `relaxed`
- Release on the store publishes the data written before it
- Acquire on the other index makes the other thread's data visible
- `alignas(64)` on both indices — without it, producer and consumer fight over one cache line
- Power-of-two size turns modulo into a mask

1.8 Common interview questions

"How would you make this class thread-safe?" Don't reach for a mutex immediately. Ask: is the computation order-dependent? Can you shard by key instead? Is one thread fast enough? A mutex is the answer only when independent work genuinely shares state.

"What's the difference between a mutex and an atomic?" An atomic makes a *single operation* indivisible. A mutex protects a *region* of code. Atomics can't protect a multi-step invariant.

"Why is `x++` not thread-safe on an `int`?" It's load, add, store. Two threads can both load the same value and both store back the same result, losing an increment.

"What is false sharing?" Two threads writing different variables that happen to sit on the same 64-byte cache line. The

line ping-pongs between cores even though there's no logical sharing. Fix with `alignas(64)` or padding. Detect with `perf c2c`.

"How do you verify a race is actually fixed?" Not by stress duration — a race made 100× rarer looks identical to one that's fixed. In order of strength: compile-time thread-safety annotations (Clang `-Wthread-safety` with `GUARDED_BY`), which is a proof over all paths; model checkers for lock-free code; ThreadSanitizer, which finds races on paths that execute; and stress testing as corroboration only, with oversubscribed threads and recorded seeds. And run every check against the *old* code first — if TSan doesn't flag the original, your harness isn't reaching the racy path. # Part 2 — Cache, Memory Layout and Language Mechanics

---

2.1 The memory hierarchy

Rough numbers, worth memorising as ratios rather than absolutes:

Level	Latency	Size
Register	0 cycles	~hundreds of bytes
L1 cache	~4 cycles (~1 ns)	32–48 KB
L2 cache	~12 cycles (~4 ns)	256 KB – 1 MB
L3 cache	~40 cycles (~15 ns)	8–32 MB, shared
Main memory	~200+ cycles (~80 ns)	GBs
NVMe SSD	~100 µs	

The single most important consequence: an L3 miss costs roughly the same as 200 arithmetic instructions. Optimising arithmetic while ignoring memory access patterns is almost always the wrong priority.

2.2 Cache lines

Memory moves in **64-byte lines**, never in smaller units. Touch one byte and you pull in 64.

This is where the Problem 2 layout bug lived:

```
        // [stock][day] layout: each stock owns a row of its history
        for (int i = 0; i < 500; ++i)
            ret[i] = close[i][t] / close[i][t-1] - 1.0;
```

You need 16 bytes from each of 500 separately-allocated rows. That's **500 cache lines fetched, 48 of every 64 bytes wasted**.

```
        // [day][stock] layout: each day owns a row of all stocks
        for (int i = 0; i < 500; ++i)
            ret[i] = today[i] / prev[i] - 1.0;
```

Two contiguous streams — **125 fully-used lines**. Measured: 1,017 ns → 339 ns, a **3× win** with no algorithmic change.

The general rule: lay data out along the axis you iterate.

2.3 AoS vs SoA

Array of Structs:

```
        struct Particle { float x, y, z, vx, vy, vz; };
        std::vector<Particle> particles;
```

Good when you touch most fields of one element together.

Struct of Arrays:

```
struct Particles {
    std::vector<float> x, y, z, vx, vy, vz;
};
```

Good when you touch **one field across many elements** — you get contiguous access and the compiler can vectorise.

If you only read `x` for a million particles, AoS wastes 20 of every 24 bytes fetched.

Interview answer: *"Depends on the access pattern. Touch most fields of one object → AoS. Touch one field across many objects → SoA, because it's contiguous and vectorisable."*

Note the assessment used AoS deliberately in Problem 1: `qty`, `avgPrice` and `notional` are *always* written together, so one struct means one lookup instead of three.

2.4 False sharing

Two threads write different variables that happen to land on the same 64-byte line. There's no logical sharing, but the cache coherence protocol doesn't know that — the line bounces between cores on every write.

```
struct Counters {
    std::atomic<long> a;   // thread 1 writes
    std::atomic<long> b;   // thread 2 writes -- same cache line!
};
```

Fix:

```
struct Counters {
    alignas(64) std::atomic<long> a;
    alignas(64) std::atomic<long> b;
};
```

Costs 56 bytes of padding each, saves constant coherence traffic. Detect with `perf c2c`.

A subtlety worth knowing: don't use `std::hardware_destructive_interference_size` for this. Its value varies with compiler version and `-mtune`, so it changes your struct layout — an ABI break from a compiler upgrade. GCC actually warns about it. Pin the constant to 64.

2.5 Container internals

`std::vector`

Contiguous. Growth is amortised O(1) by doubling. **Reallocation invalidates everything** — all iterators, pointers and references.

`reserve()` avoids that if you know the size up front.

`std::unordered_map`

An array of buckets; each bucket is a chain of separately allocated nodes. When `size / bucket_count` exceeds `max_load_factor` (default 1.0), it **rehashes**: allocates a bigger bucket array and relinks every node.

The invalidation rule you must know precisely:

	Survives a rehash?
References to mapped values	**Yes**
Pointers to elements	**Yes**
Iterators	**No**
A traversal in progress	**No**

References survive because it's **node-based** — each element lives in its own allocation that never moves. Only the bucket pointer array is rebuilt.

That distinction is exactly what makes the Problem 3 race subtle: `auto& buf = buffers_[sym]` stays valid, so the code *looks* safe. The race is in the bucket-chain walk needed to *reach* it.

And `operator[]` inserts. `map[key]` default-constructs an entry if the key is absent, so a "read" mutates the container. Use `find()` on read paths.

Complexity, and where it lies

Container	Lookup	Insert	Notes
`vector`	O(n)	O(1) amortised at end	best cache behaviour by far
`unordered_map`	O(1) average	O(1) average	pointer chasing; rehash is O(n)
`map`	O(log n)	O(log n)	red-black tree, also pointer chasing
`list`	O(n)	O(1)	terrible locality — rarely the right choice

Big-O hides constants. Linear search of a 100-element `vector` usually beats a `map` lookup, because it's one cache line at a time with perfect prefetching.

`std::sort`

Introsort: quicksort, switching to heapsort at depth `2·log n` (to guarantee O(n log n)), and insertion sort for small ranges.

It is not stable. Equal elements come out in an order determined by pivoting. That's the Problem 2 determinism bug — a quiet day puts dozens of stocks at exactly 0.00%, and their order then differs between runs.

Related: - `stable_sort` — O(n log n) but needs extra memory - `partial_sort` — top k in O(n log k) - `nth_element` — kth element in O(n), rest partitioned but unordered

Using the right one was a **2.4× win** in Problem 2, more than parallelism could offer.

2.6 Move semantics

Basic. An lvalue has a name and an address. An rvalue is a temporary. `std::move` doesn't move anything — it's a cast to rvalue reference, telling overload resolution "you may steal from this."

```
        std::vector<int> a = {1,2,3};
        std::vector<int> b = std::move(a);   // b steals a's buffer; a is now valid but unspecified
```

Rule of 0 / 3 / 5

- **Rule of 0**: if your class has no raw resources, declare none of the special members. Best default.
- **Rule of 3**: if you need a destructor, you probably need a copy constructor and copy assignment.
- **Rule of 5**: with move semantics, add the move constructor and move assignment.

A trap: declaring a destructor *suppresses* implicit move operations, so your class silently falls back to copying.

RVO and copy elision

```
std::vector<int> make() {
    std::vector<int> v = {1,2,3};
    return v;                       // NRVO: constructed directly in the caller
}
```

Since C++17 copy elision is **guaranteed** for pure prvalues.

The classic mistake:

```
return std::move(v);   // WRONG — this DEFEATS NRVO
```

It turns a named local into an rvalue reference expression, which blocks the elision. You get a move where you'd have had nothing at all. Just `return v;`.

Perfect forwarding

```
template <typename T>
void wrapper(T&& arg) {             // forwarding reference, not an rvalue ref
    inner(std::forward<T>(arg));    // preserves lvalue-ness / rvalue-ness
}
```

`T&&` in a deduced context is a **forwarding reference** (reference collapsing), not an rvalue reference.

2.7 RAII and smart pointers

RAII: acquire in the constructor, release in the destructor. The destructor runs on any exit path — normal return, early return, exception. This is why `lock_guard` exists.

	Use
`unique_ptr`	sole ownership. Zero overhead — same size as a raw pointer. **Default choice.**
`shared_ptr`	shared ownership. Reference count is **atomic**, so copies cost a `lock`-prefixed instruction. Two allocations unless you use `make_shared`.
`weak_ptr`	breaks `shared_ptr` cycles; must be `lock()`ed to use

Interview answer on cost: *"`unique_ptr` is free. `shared_ptr` copying is an atomic increment, so it's not — and in a hot loop passing `shared_ptr` by value instead of by const reference is a real cost."*

Virtual destructors: required whenever you delete a derived object through a base pointer. Otherwise it's UB — the derived destructor never runs. If a class isn't meant to be a base, don't give it a virtual destructor; it adds a vtable pointer.

2.8 Undefined behaviour you should be able to list

- Signed integer overflow (unsigned wraps, defined)
- Out-of-bounds access
- Dereferencing null or dangling
- Reading uninitialised memory
- Data races
- Violating strict aliasing (accessing an object through the wrong type)
- Violating strict weak ordering in `std::sort`
- Using a moved-from object beyond destroy-or-reassign

Why it matters practically: the compiler *optimises assuming UB doesn't happen*. A null check after a dereference gets deleted, because the dereference proved the pointer non-null.

Unsigned wraparound is defined but still a bug source:

```
size_t i = 0;
i - 1;                    // 18446744073709551615, not -1
```

That's why the ring buffer index needs `+ size` before the modulo:

```
buf[(head + size - count + i) % size];   // head - count could go "negative"
```

2.9 Common interview questions

"When is a `vector` iterator invalidated?" Any reallocation (push_back past capacity, resize, insert). `erase` invalidates from the erase point onward. `reserve` up front avoids it.

"`unordered_map::operator[]` vs `find`?" `operator[]` inserts a default-constructed value if absent, so it mutates on what looks like a read, and it's not `const`. `find` doesn't.

"What's the cost of a virtual call?" Load the vtable pointer, load the function pointer, indirect branch. Two dependent loads plus a branch the predictor can miss — but the bigger cost is that it **blocks inlining**, so all downstream optimisation is lost.

"How would you make this struct cache-friendly?" Order members largest-to-smallest to minimise padding; keep hot fields together and cold fields elsewhere; consider splitting hot/cold into separate arrays; `alignas(64)` anything written by different threads.

"`emplace_back` vs `push_back`?" `emplace_back` constructs in place from the arguments, avoiding a temporary. With an already-constructed object there's no difference. # Part 3 — Templates and Floating Point

---

3.1 Templates: why HFT codebases use them

In this domain templates exist for essentially one reason: **remove indirection from the hot path**.

A virtual call costs two dependent loads, an indirect branch, and — the real cost — it **blocks inlining**, so everything downstream of the call site loses optimisation. A template resolves the call at compile time, so it inlines and the optimiser sees through it.

```
// Runtime polymorphism: one indirect call per tick
struct Strategy { virtual void onTick(const Tick&) = 0; };
void run(Strategy* s, const Tick& t) { s->onTick(t); }

// Compile-time: inlined, no dispatch at all
template <typename Strategy>
void run(Strategy& s, const Tick& t) { s.onTick(t); }
```

CRTP (Curiously Recurring Template Pattern)

Static polymorphism — an interface with no dispatch cost.

```cpp
template <typename Derived>
class HandlerBase {
public:
    void handle(const Tick& t) {
        static_cast<Derived*>(this)->handleImpl(t);   // resolved at compile time
    }
};

class MyHandler : public HandlerBase<MyHandler> {
public:
    void handleImpl(const Tick& t) { /* ... */ }
};
```

The base knows its derived type as a template parameter, so `handle` inlines straight into `handleImpl`.

What it costs: there's no common base type, so you can't hold `vector<HandlerBase*>`. Each instantiation is separate code, so binary size and I-cache pressure grow.

`if constexpr` (C++17)

Compile-time branching. The untaken branch isn't even compiled, so it may contain code invalid for that type.

```cpp
template <typename T>
void process(T value) {
    if constexpr (std::is_integral_v<T>) {
        // only compiled when T is integral
    } else {
        // only compiled otherwise
    }
}
```

This replaced most SFINAE. Say that if asked — it shows you know the modern form.

SFINAE and concepts

SFINAE — "substitution failure is not an error." A template that fails to substitute is quietly removed from overload resolution rather than being a compile error. Used pre-C++20 to constrain templates:

```cpp
template <typename T, typename = std::enable_if_t<std::is_integral_v<T>>>
void f(T x);
```

Concepts (C++20) do the same thing readably:

```cpp
template <std::integral T>
void f(T x);
```

Know that concepts replaced SFINAE and why (error messages, readability). You don't need to write SFINAE fluently.

Type traits

Compile-time queries: `std::is_integral_v<T>`, `std::is_trivially_copyable_v<T>`, `std::is_same_v<A,B>`, `std::remove_reference_t<T>`.

The assessment used one as a **correctness guard**:

```cpp
static_assert(std::is_trivially_copyable_v<Tick>);
```

The lock-free path copies a `Tick` bitwise. If someone later adds a `std::string` field, that silently breaks — this turns it into a compile error at the line that caused it.

Variadic templates

```cpp
template <typename... Args>
void log(Args&&... args) {
    (std::cout << ... << args);   // C++17 fold expression
}
```

Know they exist and that fold expressions simplified them.

The tradeoffs — this is what gets asked

Benefit	Cost
No dispatch, full inlining	Code bloat — one instantiation per type, I-cache pressure
Type-safe compile-time config	Slow compiles, everything in headers
Optimiser sees through everything	Unreadable errors (much better with concepts)
Zero abstraction penalty	**Cannot read the parameter from a config file**

That last one is the practical one. In Problem 2 the window size was deliberately a *runtime* `Config` field rather than a template parameter, precisely so it could be configured per deployment.

The question to expect: *"Show me something you templated and tell me why."* Have one you'd defend, and ideally one you'd now write differently — admitting a template that should have been a runtime parameter is a strong answer, because it shows you weigh the tradeoff instead of defaulting to the clever tool.

3.2 Floating point

You now have a genuinely strong, measured story here. Make sure the fundamentals underneath it are solid.

IEEE-754 doubles

64 bits: 1 sign, 11 exponent, 52 mantissa. Gives ~15–17 significant decimal digits.

Not every decimal is representable. `0.1` is a repeating fraction in binary, so:

```cpp
0.1 + 0.2 == 0.3      // false
```

`0.1 + 0.2` is `0.30000000000000004`.

Consequence: never compare floats with `==` for values that came from arithmetic. Compare against a tolerance, or use exact integers.

Exception: comparing a value you **copied** rather than computed is fine — that's why the assessment's parity tests use exact `==` and hex comparison. There the point is that the bits must be identical.

NaN

Produced by `0.0/0.0`, `inf - inf`, `sqrt(-1)`.

The rule that matters: every ordered comparison involving NaN is false.

```cpp
NaN == NaN    // false
NaN != NaN    // TRUE  (the standard NaN test)
NaN < 5       // false
NaN > 5       // false
NaN <= 5      // false
```

Two practical consequences used throughout the assessment:

```cpp
if (!(price > 0.0))       // catches zero, negatives, AND NaN
if (price <= 0.0)         // MISSES NaN — because that's also false
```

and:

```
if (!(std::fabs(r) <= band))   // excludes NaN
if (std::fabs(r) > band)       // ADMITS NaN
```

Equivalent for normal numbers, opposite for NaN. Choosing deliberately is the mark of someone who's been bitten.

NaN in `std::sort` is UB, not just a bad order. It breaks strict weak ordering: with NaN, `a` looks equivalent to `b` and `b` to `c`, but `a` and `c` are not equivalent. Transitivity fails, and introsort's internal logic depends on it. Measured effect: 72 NaNs in 500 elements left **25 perfectly valid entries out of order**.

Infinity

`1.0/0.0` gives `+inf`. Unlike NaN, **infinity compares consistently** — which makes it more dangerous in a ranking. A stock with a zero prior close produces `+inf` and sorts cleanly to rank 1. No crash, no warning, and the signal ships.

FMA and `-ffp-contract`

A fused multiply-add computes `a*b + c` with a **single rounding** instead of two. It's *more* accurate — which is exactly why it breaks bit-for-bit reproduction of a two-step computation.

Measured on the assessment, 3,000 fills against the Python:

Build	Fills differing
`-O2`	**0**
`-O2 -ffp-contract=off`	**0**
`-O2 -march=native`	**1,781**
`-Ofast -march=native`	**1,659**

The assembly:

```
-O2                              mulsd, mulsd, addsd      (two roundings)
-O2 -march=native                vmulsd, vfmadd132sd      (one rounding)
-O2 -march=native -fp-contract=off vmulsd, vmulsd, vaddsd   (two again)
```

The subtlety worth knowing: GCC's default is already `-ffp-contract=fast`. Plain `-O2` passes only because **baseline x86-64 has no FMA instruction** — the compiler is allowed to fuse but has nothing to fuse with. Add `-march=native` and it appears. And **FMA is baseline on ARMv8**, so the same code would contract by default on Graviton or Apple Silicon.

So parity survives on two coincidences — target ISA and optimisation level — neither of which anyone chose deliberately. Setting the flag explicitly makes it depend on a stated decision.

`-ffast-math`

Enables a bundle: assume no NaN/inf, allow reassociation, flush denormals, allow reciprocal approximation. **Breaks IEEE compliance.** Never use it where reproducibility or NaN handling matters.

Denormals

Numbers very close to zero, below the normal exponent range. On many CPUs operations on them trigger a **microcode assist costing ~100 cycles**. In a hot loop that's a cliff. Fix by enabling flush-to-zero (FTZ/DAZ), or by using fixed-point integers so the case can't arise.

Fixed point as the alternative

For prices that are exact multiples of a tick, integers are strictly better:

```
int64_t price_ticks = round(price_rupees * 10000);   // 4 decimal places
```

- Exact under addition and subtraction — no drift, ever
- No NaN, no infinity, no denormals
- Faster than floating point

Cost: you must manage scale yourself, and a units error becomes a silent 10,000× rather than a visible one. Division still

rounds, so you decide where the remainder goes.

Measured drift for comparison: 200,000 accumulating float additions of `1/3` drifted from `0.3333333333333333` to `0.33333333333395226`. Fixed point: exact.

3.3 Common interview questions

"Why is `0.1 + 0.2 != 0.3`?" Neither 0.1 nor 0.2 is exactly representable in binary; the sum's rounding error lands just above 0.3.

"How do you compare two floats?" Depends on intent. Values from arithmetic: relative tolerance. Values that must be bit-identical (a copy, a serialisation round-trip): exact `==`, or compare `.hex()`.

"What does `-ffast-math` do and why avoid it?" Allows reassociation, assumes no NaN/inf, flushes denormals. Faster but not IEEE-compliant, so results become compiler- and flag-dependent.

"When would you use a template instead of a virtual function?" When the type is known at compile time and the call is hot enough that inlining matters. Cost is code bloat and losing runtime configurability.

"What does CRTP solve?" Static polymorphism — an interface-like structure with dispatch resolved at compile time, so it inlines. Cost: no common base type, so no heterogeneous container. # Part 4 — Low Latency Systems and Python

---

4.1 The tick-to-trade path

The end-to-end chain, and roughly where the time goes:

1. Exchange sends a packet
2. NIC receives it
3. Kernel or bypass stack delivers it to userspace
4. Decode the wire protocol
5. Strategy decides
6. Encode an order
7. Send it back out

Where the time actually is: the network stack and the NIC dominate unless you've already bypassed them. Application logic is usually a small fraction. That's why kernel bypass is the first thing anyone does — optimising your decode loop while going through the kernel is optimising the wrong 5%.

Same lesson as Problem 2: measure before you optimise, and check what fraction of the total you're actually addressing.

4.2 Kernel bypass

The kernel network stack costs ~10 µs and is unpredictable — context switches, interrupts, copies. Bypass gets the packet from the NIC to userspace directly.

- **DPDK** — poll-mode drivers, userspace networking, huge pages
- **Solarflare / Onload** — `LD_PRELOAD` shim, so existing socket code works unchanged
- **Exablaze / Cisco Nexus SmartNIC** — hardware-level, sub-microsecond
- **io_uring** — not bypass, but far less syscall overhead than epoll

Busy-poll vs interrupt: interrupts are efficient but add latency and jitter. Busy-polling burns a core to avoid the wake-up. In HFT you burn the core.

4.3 The rules for a hot path

State these as absolutes; they're what the discipline actually amounts to.

No allocation. `malloc` may take a lock and may fault in a page. Preallocate everything; use object pools or ring buffers.

No locks. A descheduled thread holding a lock stalls everyone behind it, unboundedly.

No syscalls. Each is ~1 µs plus a mode switch.

No logging that formats. Push binary records into a lock-free queue; a low-priority thread formats them later. An `iostream` call in a nanosecond function costs orders of magnitude more than the function.

No exceptions on the happy path. Throwing is expensive (unwinding, RTTI). Zero-cost only when not thrown.

No branches you can avoid, and mark the ones you can't: `[[likely]]` / `[[unlikely]]`.

Warm everything. Pre-fault pages, run a keep-warm loop before the open. A page fault or cold L2 on the first tick of the day is a multi-microsecond outlier at the worst moment.

4.4 System tuning

Technique	What it does
CPU pinning (`taskset`, `sched_setaffinity`)	thread stays on one core, so its cache stays warm
`isolcpus`	kernel schedules nothing else on those cores
NUMA locality	allocate on the node running the thread; cross-socket access is ~100 ns
Huge pages (2 MB)	fewer TLB misses on a large working set
Disable frequency scaling	prevents the CPU dropping clock and adding jitter
Disable hyperthreading	siblings share L1/L2 and execution ports
IRQ affinity	keep interrupts off your trading cores

4.5 Measurement

Timing: `rdtsc` / `rdtscp` — a few cycles, versus ~20–25 ns for `clock_gettime`. `rdtscp` serialises; `rdtsc` alone can be reordered.

Always report percentiles, never a mean. p50, p99, p99.9, max. A mean hides exactly the tail that matters, and in trading the tail *is* the risk.

`perf` commands worth knowing:

```
perf stat ./prog          # IPC, cache misses, branch misses -- start here
perf record ./prog        # attribute time by function
perf report
perf c2c record ./prog    # cache-line contention and false sharing
perf mem record ./prog    # where memory accesses land
```

The three numbers from `perf stat` that immediately tell you the shape of a problem: **IPC** (low = stalling), **LLC-load-misses** (memory bound), **branch-misses** (mispredict bound).

In Problem 2 those pointed straight at the sort, which was mispredict-bound — which is why `partial_sort` helped and parallelising the arithmetic didn't.

Sanity habit: check that your component timings add up to your end-to-end number. When they don't, the benchmark is usually lying. That mismatch is how the layout bug in the assessment's carried state was found.

4.6 Benchmarking traps

Three that fired during the assessment, all of which made the code look *better* than it was:

Constant input. Feeding the same data repeatedly made the engine look 4× faster, because the sort's input arrived already sorted.

Hot repeated data. Re-sorting the same array in a loop looked 5× faster, because the branch predictor memorised the comparison pattern. Cycling through 256 distinct datasets revealed the truth.

Dead code elimination. If you don't consume the result, the compiler deletes the work. Use a `volatile` sink or `benchmark::DoNotOptimize`.

Also: run each measurement multiple times and take the **minimum**, not the mean — the minimum is the least contaminated by scheduling noise.

Python

4.7 The data model

```python
class Position:
    __slots__ = ('qty', 'avg_price')      # no __dict__, less memory, faster attribute access

    def __init__(self, qty, avg_price):
        self.qty = qty
        self.avg_price = avg_price

    def __repr__(self):
        return f"Position(qty={self.qty}, avg={self.avg_price})"

    def __eq__(self, other):
        return self.qty == other.qty and self.avg_price == other.avg_price

    def __hash__(self):                    # define alongside __eq__ or the object is unhashable
        return hash((self.qty, self.avg_price))
```

Know: `__init__`, `__repr__` vs `__str__`, `__eq__`/`__hash__` pairing, `__enter__`/`__exit__` for context managers, `__iter__`/`__next__`.

`__slots__` is worth mentioning — it removes the per-instance `__dict__`, saving memory and speeding attribute access. Relevant when you hold millions of objects.

4.8 The GIL

What it is: one lock protecting the interpreter. Only one thread executes Python bytecode at a time.

What it means:

Workload	Threads help?
CPU-bound pure Python	**No** — the GIL serialises it
I/O-bound	**Yes** — the GIL is released during I/O
NumPy / C extensions	**Yes, if the extension releases it**

Releasing it from C++:

```cpp
{
    py::gil_scoped_release release;   // RAII: reacquired at the closing brace
    heavy_cpp_work();                 // must touch NO Python objects
}
```

Safe only because nothing inside touches a `py::` object. Touching one after release is a **crash**, not a slowdown.

That's also why a list-of-tuples batch API can't release the GIL — every element is a Python object unpacked inside the

loop.

`multiprocessing` sidesteps the GIL with separate processes, at the cost of IPC and pickling.

Note: Python 3.13 introduced an experimental free-threaded build. Worth knowing it exists.

4.9 The `dict.get` vs `[]` trap

```
d[key]              # raises KeyError if absent
d.get(key)          # returns None
d.get(key, 0)       # returns a default
d.setdefault(key, 0)  # returns AND INSERTS
```

Note this is the *mirror image* of C++: in Python `[]` raises, and it's `setdefault` that inserts. In C++ `operator[]` silently inserts. Both traps, opposite directions.

4.10 NumPy

Vectorise. A Python loop over an array is ~100× slower than the vectorised form, because each iteration is interpreter overhead.

```
result = (today / prev) - 1.0        # C speed
result = [t/p - 1 for t, p in zip(today, prev)]   # ~100x slower
```

Views vs copies. Slicing gives a **view** sharing memory; fancy indexing gives a **copy**.

```
a[1:5]       # view -- modifying it modifies a
a[[1,2,3]]   # copy
```

Broadcasting. Shapes align from the right; dimensions of 1 stretch.

Structured arrays are strided. A field view of a structured array has a stride equal to the **full record size**, not the field's itemsize. Walking it with `base + i * itemsize` reads interleaved bytes from neighbouring fields — this was a real bug in the assessment's bindings, and it produced plausible-looking garbage rather than an obvious failure.

Also: NumPy `'S'` string fields are NUL-**padded**, not NUL-terminated. A symbol that exactly fills the field has no terminator.

Floats are IEEE-754 doubles, same as C++. Use `.hex()` to compare exactly.

4.11 pandas

- `groupby().agg()`, `merge` (know the join types), `pivot`
- `resample` for time series
- **Vectorise; avoid `iterrows`** — it's row-by-row Python
- `.loc` (label) vs `.iloc` (position)
- `SettingWithCopyWarning` means you modified a slice that may be a copy
- `df.values` gives the underlying NumPy array

4.12 pybind11

The one lesson that matters, measured over 200,000 fills:

Path	ns/fill	Speedup
Pure Python	545	1.0×
C++, one call per fill	**493**	**1.1×**
C++, list of tuples	316	1.7×
C++, batch arrays, GIL released	**104**	**5.3×**

A per-fill binding gets 1.1× on a core that is 45× faster. The entire gain is handed back in boundary-crossing overhead.

The pattern: **convert at the boundary once, then release the GIL and do the whole loop with no Python involvement.**

Two gotchas from actually building it:

- pybind11 converts default arguments **eagerly at registration time**, so a type used as a default must already be registered — otherwise the module fails at **import**, not at call.
- `py::value_error` becomes a Python `ValueError`; `std::invalid_argument` becomes a generic `RuntimeError`.

4.13 Common Python interview questions

"Why doesn't threading speed up my CPU-bound Python?" The GIL. Use multiprocessing, or push the work into C.

"List vs tuple?" Mutable vs immutable; tuples are hashable so they can be dict keys.

"What's a generator?" A lazy iterator — computes on demand, so memory is O(1) rather than O(n).

"Shallow vs deep copy?" Shallow copies the container but shares the elements; deep copies recursively.

"How would you speed up slow Python?" Profile first (`cProfile`, `line_profiler`). Then: vectorise with NumPy, or move the hot part to C++ **with a batch interface**. # Part 5 — OOP Design and Quant Fundamentals

---

5.1 SOLID, briefly

Two of the five actually get asked about; know those properly and the rest by name.

S — Single Responsibility. One reason to change. The Problem 1 tracker tracks positions; it doesn't decide what a bad price is (that's `checkFill`) and doesn't invent a delta (the caller supplies it).

O — Open/Closed. Open for extension, closed for modification.

L — Liskov Substitution. *This one gets asked.* A derived class must be usable anywhere the base is, without the caller knowing. The classic violation: `Square` deriving from `Rectangle`. Setting width on a Rectangle shouldn't change height — but for a Square it must, so it breaks the base's contract.

I — Interface Segregation. Many small interfaces beat one fat one.

D — Dependency Inversion. *Also asked.* Depend on abstractions, not concretions. Practically: inject the dependency rather than constructing it internally, so it can be swapped in tests.

5.2 Composition over inheritance

The fragile base class problem: a change to the base silently breaks derived classes, because inheritance exposes implementation, not just interface. Deep hierarchies make it worse.

Prefer: - **Has-a over is-a.** A `PositionTracker` *has* a map; it isn't one. - **Interface inheritance over implementation inheritance.** Inherit from pure abstract bases. - **Never inherit from STL containers** — they have no virtual destructor, so deleting through a base pointer is UB.

Interview answer: *"Inheritance couples you to the base's implementation, so a base change can break derived classes silently. Composition keeps the coupling explicit. I inherit for interfaces, compose for behaviour."*

5.3 Virtual functions and destructors

```
class Base {
public:
    virtual ~Base() = default;      // REQUIRED if deleting through Base*
    virtual void doWork() = 0;      // pure virtual -> abstract
};
```

Rules:

- Virtual destructor whenever a class is deleted polymorphically. Otherwise UB — the derived destructor never runs.
- **Don't** add one if the class isn't a base; it forces a vtable pointer into every object.
- Use `override` — it catches signature mismatches at compile time.
- `final` prevents further overriding and lets the compiler devirtualise.

Cost of a virtual call: load the vtable pointer, load the function pointer, indirect branch. Two dependent loads plus a possible mispredict — but the real cost is that it **blocks inlining**.

Alternatives: CRTP (compile-time), `std::variant` + `std::visit` (closed set, no heap), function pointers or `std::function` (the latter may allocate).

5.4 PIMPL

```
// header
class Widget {
public:
    Widget();
    ~Widget();                      // must be defined in the .cpp
    void doThing();
private:
    struct Impl;
    std::unique_ptr<Impl> pimpl_;
};
```

Buys: compilation firewall (changing the implementation doesn't force a rebuild of every user), stable ABI, hidden dependencies.

Costs: a heap allocation and a pointer indirection per call — so **not** for hot-path types.

The destructor must be defined in the `.cpp` where `Impl` is complete, otherwise `unique_ptr` can't destroy an incomplete type.

5.5 Common design questions

"How would you design a market data feed handler?" Separate the layers: wire decoding, book building, subscriber dispatch. Feed thread owns the book; readers get consistent snapshots. Preallocate everything. No locks on the tick path.

"How do you make this testable?" Inject dependencies rather than constructing them. Keep the pure logic separate from I/O. In Problem 2 the ranking kernel is a free function taking raw pointers, so it's testable without any file or clock.

"When would you use a singleton?" Rarely — it's global state with extra steps, hides dependencies, and makes tests order-dependent. Prefer passing the thing in.

Quant Fundamentals

Given your background is infrastructure rather than derivatives, this is the section with the most genuinely new material. Prioritise being able to explain over being able to derive.

5.6 Options: the basics

Call: the right, not the obligation, to buy at strike `K`. Worth `max(S − K, 0)` at expiry. **Put:** the right to sell at `K`. Worth `max(K − S, 0)`.

The `max(·, 0)` is the whole subject. It bends the payoff at `K`, and everything interesting — volatility sensitivity, gamma, the smile — comes from that kink.

Buyer: limited loss (the premium), unlimited gain. **Seller:** limited gain (the premium), unlimited loss. That asymmetry is why sellers are margined and buyers aren't.

Moneyness with spot at 48,500:

	Call	Put
ITM	K < 48,500	K > 48,500
ATM	K ≈ 48,500	K ≈ 48,500
OTM	K > 48,500	K < 48,500

Intrinsic = what you'd get exercising now. **Time value** = everything above that. An ATM option is all time value, and time value decays to exactly zero at expiry.

Indian index options specifically: European (exercise only at expiry), cash-settled, premium paid upfront. All three matter for parity.

5.7 Put-call parity

The only model-free relationship in options. Long a call and short a put at the same strike pays `S − K` on **every** path — the kinks cancel — so it *is* a future.

$$C - P = (F - K)\,e^{-rT} \qquad \Rightarrow \qquad F_{syn} = K + (C - P)\,e^{+rT}$$

The sign of the exponent: premium is paid **today**, futures P&L settles at **expiry**, so the premium difference is carried **forward**. Writing `e^{-rT}` there is the standard mistake.

Synthetics:

Want	Build from
Long future	long call + short put
Short future	short call + long put
Long call	long future + long put
Long put	short future + long call

Structures:

- **Conversion** — buy the future, sell the synthetic. Do it when the synthetic is rich.
- **Reversal** — sell the future, buy the synthetic. When the synthetic is cheap.
- **Box** — a call spread plus a short put spread at two strikes. Pays exactly $K_2 - K_1$ at expiry, whatever happens. A synthetic zero-coupon bond made of options.

Why parity is special: Black-Scholes needs a distribution, constant vol, continuous trading. Parity needs none of it — it's two portfolios with the same payoff. If parity breaks, *someone is giving away money*, not "the model is wrong."

5.8 The Greeks

Greek	Meaning	Intuition
Delta	$\partial V/\partial S$	How many shares the option behaves like. ~0.5 ATM. Roughly the probability of finishing ITM.
Gamma	$\partial^2 V/\partial S^2$	How fast delta changes. Peaks ATM near expiry.
Vega	$\partial V/\partial \sigma$	Sensitivity to implied vol. **Identical for a call and a put at the same strike.**
Theta	$\partial V/\partial t$	Time decay. Negative if long, positive if short.
Rho	$\partial V/\partial r$	Rate sensitivity. Usually the least interesting.

Gamma is the one to understand properly. Being **short gamma** means your delta moves against you: as the price rises your delta goes more negative, so you keep buying high and selling low to stay hedged. That's the market maker's structural risk — you collect theta every day and lose it in one big move.

Long gamma is the reverse: you buy low and sell high rehedging, but you pay theta for the privilege.

Why vega is identical for call and put: it falls straight out of parity. `C − P` doesn't depend on σ, so `∂C/∂σ − ∂P/∂σ = 0`.

Why the conversion is safe: short call + long put has net vega of exactly zero, so an IV move after the trade is on can't touch the locked P&L.

5.9 Implied volatility

IV is a quoting convention, not a forecast. Given a market price, what σ reproduces it in Black-Scholes? It strips out strike, expiry and forward so different options become comparable — the same way a bond yield is more useful than a bond price.

Smile / skew: Black-Scholes assumes one σ for all strikes. Reality doesn't: real returns have fat tails and crashes are faster than rallies, so OTM puts are bid up. Plot IV against strike and you get a smile or a skew.

The crucial distinction: the smile is about *different strikes*. Parity is about the *call and put at the same strike*. Whatever shape the smile has, the call and put at one strike must imply the **same** vol — because their price difference is pinned by parity, which doesn't know what vol is.

So a call/put IV gap at one strike **isn't smile — it's a parity violation.** In the assessment: 72.5 bps gap at one strike, under 1 bp at the neighbours. And repricing off the synthetic forward collapsed it to exactly zero, proving the IV gap and the forward dislocation are the same fact in different units.

5.10 P&L and exposure vocabulary

The Problem 4 lesson: a mislabelled number on a risk blotter is a real incident.

Term	Definition
Realised P&L	Banked when a fill reduces the position. Permanent. Mark-independent.
Unrealised (MTM) P&L	Open position at a chosen mark, minus cost. Re-prices every tick.
Gross traded notional	$\Sigma\lvert qty\rvert \times$ price over all fills. Turnover. **Only ever increases.**
Net notional exposure	Signed qty × mark. The directional number.
Cost basis	What the open position cost — not what it's worth.
Delta exposure	Position delta × mark. For cash equity, delta is exactly 1 per share.

Cost-basis convention matters. Weighted average vs FIFO vs LIFO give different realised/unrealised **splits** — but the **total is always the same**. In India capital gains on listed equity are computed FIFO, so a weighted-average risk book will correctly disagree with the tax book. Reconcile on total, not the split.

Why a market maker watches net delta, not gross notional: their business is enormous gross volume with almost no directional view. They can turn over crores and finish delta-flat. Gross notional then describes their *activity* and their costs; net delta describes the only thing that loses money on a gap — the leftover inventory.

5.11 Execution quality

Benchmark	Meaning	Gameable how
Arrival price	Price when the decision was made	Punishes you for market moves outside your control
Interval VWAP	Volume-weighted average over the window	Rewards trading slowly even when the market runs away
TWAP	Time-weighted average	Ignores volume entirely
Close	Closing price	Suits index funds, almost nobody else

Slippage is undefined without naming a benchmark. “Our slippage was zero” is not a statement.

Implementation shortfall = the full cost from decision to completion: market impact, timing, opportunity cost of the unfilled part.

Matching VWAP means you were average, not free. Even a perfect VWAP fill has non-zero shortfall whenever the market moved between decision and execution.

5.12 Indian market specifics

- **Lot sizes**: F&O trades in lots (a fixed multiple of shares), revised periodically by NSE. Cash equity has no lot size — calling shares “lots” overstates by the whole multiple.
- **Circuit limits**: typically 20% for most stocks; index moves trigger market-wide halts.
- **STT**: charged on the sell side for delivery, and **on exercise of ITM options, on intrinsic value**. That last one is a genuine trap for held-to-expiry arbitrage — the cost scales with where the index settles, which you don’t know when you put the trade on.
- **SPAN margining**: portfolio-based, so it recognises hedges. A conversion needs far less margin than the gross legs.
- **Expiry**: index options are weekly and monthly; settlement uses an averaging window rather than a single print, specifically to make it hard to manipulate.
- **NSE protocols**: NEAT / NOW for order entry; multicast market data feeds.

5.13 Common quant interview questions

“What’s delta hedging?” Holding an offsetting position in the underlying so the position’s value doesn’t move with small price changes. It has to be re-done as delta changes — that’s gamma.

“Why does a market maker care about gamma?” Short gamma means rehedging always buys high and sells low. You collect theta daily and lose it in one big move.

“What does implied vol mean?” The vol that reproduces the market price in a model. A quoting convention, not a prediction.

“How do you know if an option is mispriced?” Relative to what? Against parity you can say something model-free. Against a smile fit you need a model, and then you’re expressing a view rather than arbitraging.

“Realised vs unrealised P&L?” Realised is banked on a close and can’t change. Unrealised is the open position at a mark and re-prices constantly.